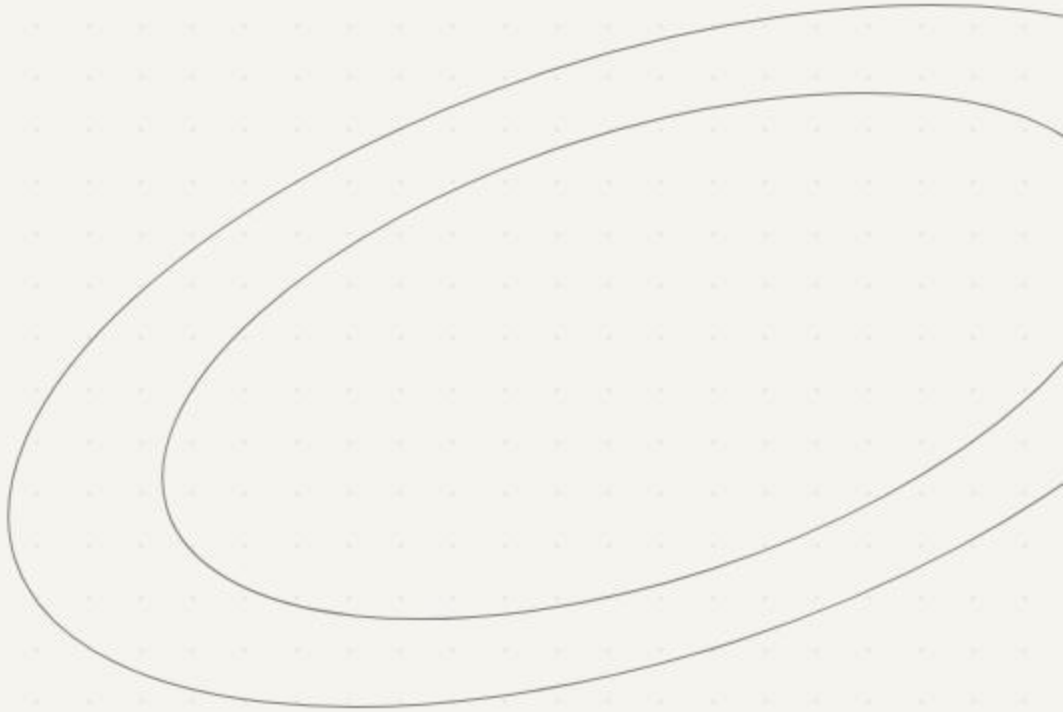
天体力学 × AI · 2026

从下一个 token 到下一个定理

一个模型如何工作，以及我如何使用它



刘博文

From Next Token to Next Theorem

模型看到的，是 10 个整数

EN · 一句话

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 个字符 → **10** 个 token → 10 个整数 · _ 表示前导空格

中文 · 《天体力学基础》内容简介

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 个字符 → **10** 个 token → 10 个整数 · 系统 / 介绍 各占一个 token

01

这串整数是怎么来的?

02

它怎么预测下一个?

03

我们怎么用它干活?

切分是查表的结果

三体问题 is a classic problem in celestial mechanics.

三	体	问题	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
3236	7074	28917	382	261	13686	4792	306	134437	52347	13

_ 表示前导空格 · Celestial → Cel + estial

≈5
字符 / 英文单词
含空格，常见值

≈4
字符 / token
英文经验值

≈1.3
token / 英文单词
≈ 5 ÷ 4

5–20 万
常见词表大小
50k – 200k

为什么 字母级：信息少，序列长 4–5 倍 → 词级：词表爆炸（百万级） → **子词：折中**

GPT-2 50k · GPT-4 100k · Llama 3 128k · GPT-4o 200k

> 子词不是语义单位，是**序列长度**与**词表大小**之间的工程折中。

英文：1 token ≈ 4 字符 ≈ 0.75 词

词表：一张固定的映射表

《天体力学基础》 · 行号 = ID · 按 ID 排序

ID	片段	
867	天	
7074	体	
8590	学	
11343	力	
20121	书	
31936	系统	
78580	基础	← 本次查询
199999	<|endoftext|>	特殊

只展示 8 行 共 200,019 行

词表大小 = 行数

200,019

o200k · GPT-4o 代

- **0**
- **50k** GPT-2
- **100k** GPT-4 代
- **128k** Llama 3
- **200k** GPT-4o 代

200,019 × 4,096 ≈ **8.2 亿参数**

> ID 是**行号**；换模型，就是换一张表。

Token 是模型的单位，也是账单的单位



上月用量 · 平均每次 ≈ 110k token · ≈ 1 分钱

≈ 0.3 token

1 个英文字符

≈ 0.6 token

1 个中文字符

直观理解：一个词、一个数字、一个符号 ≈ 1 token；不同 tokenizer 不同，以返回的 usage 为准。

> Token 既是模型的最小单位，也是账单的最小单位。

一次调用，钱花在哪

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

deepseek-flash · 1M 上下文 · 384K 输出 · 元 / 百万 token

1M 上下文 ≈ 一部《三体》三部曲

一次调用 · 空闲时段

输入 100k + 输出 2k

¥0.01 缓存命中

¥0.11 缓存未命中

> 同一段上下文，命中缓存与不命中，差一个**数量级**。

文字先变成 ID，再变成向量

片段

问题

切分后的单位

→

ID

28917

词表第 28917 行

→

向量

0.12 −0.44 0.08 …

4096 维 · 训练得到

> ID 是**地址**，向量是**内容**。

这一步叫 embedding：每一维没有名字，但距离有意义。

模型的工作：输入 → 概率分布

输入 token

从 | 下一个 | token | 到 | 下一个

每个 token 已经是一个向量

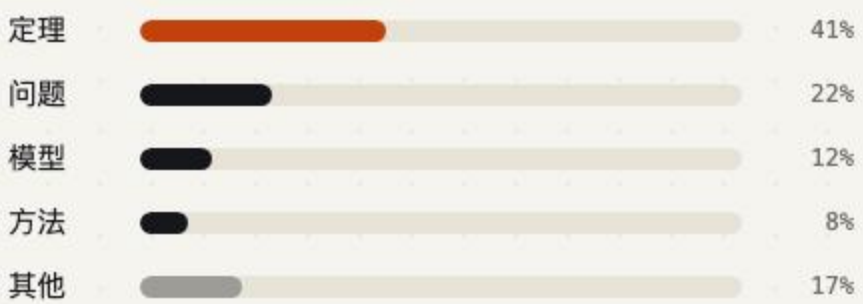
模型

Transformer

N 层 · 自注意力

GPT · Llama · Qwen 都是变体

softmax(z)



$$z = f_\theta(x_{1:t}) \in \mathbb{R}^{|V|} \qquad p_\theta(y \mid x_{1:t}) = \frac{\exp(z_y)}{\sum_{v \in V} \exp(z_v)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

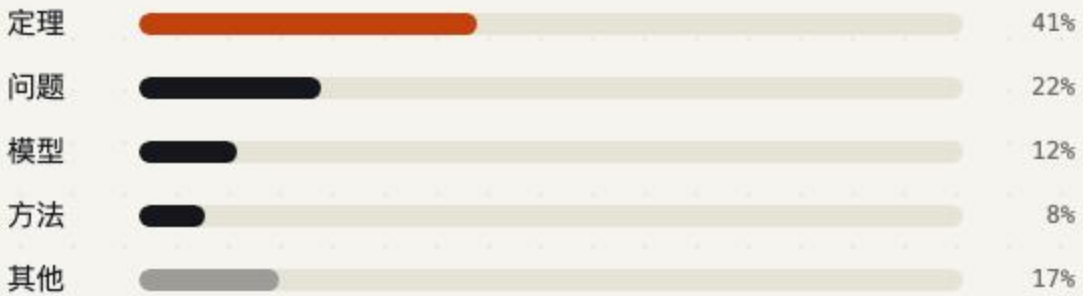
> 它不查数据库，也不“理解”；它给词表里每个 token **打分**，再把分数变成**概率**。

采样：从分布里取一个

概率分布 · 前文

“一个重要的……”

$$y_{t+1} \sim p_\theta\left(\cdot \mid x_{1:t}\right)$$



软件算法 · 随机性

T > 0 主动采样（掷骰子）；**T = 0** 贪心取最高概率

硬件物理 · 数值扰动

GPU 浮点不满足结合律：并行累加顺序随机 · 批处理调度抖动

自回归放大

哪怕 T = 0，临界点微小扰动选错 token，后续轨迹全面分叉

输入 → Transformer → 概率分布 → 采样 / 临界决胜 → 接回输入 ↺

输出随机来自两层：**软件的主动采样（温度）**，与 **GPU 并行的浮点累加扰动**。

采样：在累积分布上扔一个随机数

token	p	F(y)
定理	0.41	0.41
问题	0.22	0.63
模型	0.12	0.75
的	0.08	0.83
其他	0.17	1.00



u = 0.57 ∈ [0.41, 0.63) → 取「问题」

TEMPERATURE

$$p_i^{(T)} = \frac{p_i^{1/T}}{\sum_j p_j^{1/T}}$$

T 小 → 更尖锐；T 大 → 更平坦

TOP-P

$$\min |S| \text{ s.t. } \sum_{i \in S} p_i \geq 0.9$$

截掉长尾，再归一化

温度改形状，top-p 截尾巴，**u** 决定落在哪一段。

模型没有记忆

调用 A

system	你是天体力学助手
user	希尔区是什么?
assistant	希尔区是……
user	它为什么重要?
assistant	因为它界定了……

调用 B · 新的一次

system	你是天体力学助手
user	它为什么重要?
assistant	（不知道“它”指什么）

> 模型无状态；所谓“对话”，是客户端把历史**重发**一遍。

参数在推理时不更新；状态只存在于上下文里。

预测下一个词，怎么就变成了做题?

PRETRAIN

知识

读遍语料，学会语言与世界

SFT

行为

看示范，学会按指令回答

RL

策略

做题对答案，学会搜索与纠错

换的是**训练信号**。

Pretrain：把互联网读一遍

$$L(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

给定前文，预测下一个 token

数据

万亿 token

网页、代码、论文、书

算力

千–万卡

月级训练

产出

base model

会续写，不会回答问题

> 它学会的是世界的**统计结构**。

next-token prediction

前 20 个 token，19 个样本

《天体力学基础》· 内容简介

本书系统地介绍了天体力学的基本概念和定理，以及一些比较有代表性的研究成果。全书共五章：第一章二体问题，第二章限制性三体问题，第三章受摄二体问题，第四章天体运动中的共振现象，第五章保守系统中的有序与混沌运动。本书论述严谨，深入浅出，具有天体力学、分析力学和现代数学基础的读者可以较流畅地阅读。
本书可作为高等学校天文学类专业的教材，也可供其他相关专业选用，还可作为天文学工作者的参考书。

前 20 个 token

本 | 书 | 系统 | 地 | 介绍 | 了 | 天 | 体 | 力 | 学 | 的 | 基本 | 概 | 念 | 和 | 定 | 理 | ，以及 | 一些 | 比较

输入（前缀）	目标
本	书
本 书	系统
本 书 系统	地
本 书 系统 地	介绍
本 书 系统 地 介绍	了
本 书 系统 … 一些	比较

19

个训练样本

每个位置都预测下一个 token

$$L = -[\ \log p(书|本) + \log p(系统|本\ 书) + \ldots + \log p(比较|前\ 19\ 个)\ ]$$

> 训练并行，推理串行。

pretrain

这个 loss 是什么

负对数似然 · 交叉熵

P(正确 TOKEN)	−LOG P
1.00	0
0.50	0.69
0.10	2.30
0.01	4.61

单位：nats · ÷ ln 2 = bits

困惑度 PPL = exp(平均 −log p)

∂L/∂z = p − onehot(y)

单个位置

−log p = 惊讶度，也是编码这个 token 的长度

整句

相加 = 编码整句的总长度；平均后取 exp，就是困惑度

最小化

= 最大似然 = 最优压缩

> 这个 loss 就是模型给数据**编码的长度**。

压缩即智能

要预测得准，
就必须把世界**压进参数**里。

预测

解释每个 token

不能跳过任何细节

压缩

挤掉冗余

留下可复用的结构

结构

语法 · 因果 · 物理

迁移到没见过的题

压缩解释知识从哪来；搜索与纠错，靠后面的 RL。

> 这也是我近几年最相信的一句话。

SFT：看标准答案学做事

supervised fine-tuning · 监督微调 / 指令微调

system	你是《天体力学基础》的助教。	输入
user	这本书讲什么?	输入
assistant	系统介绍天体力学的基本概念和定理，以及一些有代表性的研究成果。	目标

橙色 = 唯一参与 loss 的 token。

SFT $-\Sigma_{t \in answer} \log p(y_t \mid prompt, y_{<t})$

数据

（指令, 标准答案）对

目标

还是下一个 token；条件是 system + user + 已答部分

边界

行为克隆：模仿示范；知识来自预训练；难题不一定做对

> 同一套 loss，换了一个**求和范围**。

一条消息，三种角色

SYSTEM
你是《天体力学基础》的助教，只回答本书内容。

USER
第四章讲什么?

ASSISTANT
天体运动中的共振现象。

SYSTEM
身份、规则、边界；由开发者写

USER
任务与材料；由用户写

ASSISTANT
模型的回答；训练时只对它算 loss

system / user = 条件 · assistant = 监督目标（只算这些 token 的 loss）

> system 定规则，user 给任务，assistant 是学习目标。

它到底怎么预测

已生成	候选（概率）	采样
（空）	系统 .62 · 本书 .11 · 天体 .08	系统
系统	介绍 .48 · 是 .12 · 主要 .09	介绍
系统介绍	天 .71 · 本 .06 · 了 .05	天
系统介绍天	体 .66 · 文 .12 · 气 .08	体
系统介绍天体	力 .58 · 学 .21 · 的 .07	力
系统介绍天体力学	的 .44 · 学 .18 · 是 .09	的

概率为示意，用于说明机制 · 真实输出以模型为准

01 算概率

给定已生成的全部 token

02 采样

按概率取一个

03 接回

拼到输入末尾，重复

> 一次只走一步；把结果接回输入，再走下一步。

什么是强化学习

在环境里试错，用奖励信号调整策略，让长期回报最大。



没有标准答案

只有奖励

不告诉你该做什么，只告诉你结果好不好

试错

动作 → 反馈

好就强化，坏就改

目标

累积奖励

目标是长期回报

目标 · 最大化期望折扣回报

$$\max_{\pi} E\left[\sum_t \gamma^t r_t\right]$$

π 策略 · γ 折扣 · r_t 奖励 · E 期望

> RL 学的是**策略**。

agent · environment · reward

RL 的老战场：先打赢游戏

2013

Atari · DQN

从像素学会打游戏

2016

AlphaGo

自我对弈 + 树搜索

2018

Dota 2 · OpenAI Five

长时程、团队协作

2019

StarCraft II · AlphaStar

不完全信息、大动作空间

- 有 simulator
- 有 reward
- 可无限试错

DeepMind：Atari · AlphaGo · AlphaStar（RL） | AlphaFold → 2024 诺贝尔化学奖（Hassabis & Jumper）

> **AlphaFold 是监督学习。** 它用已知结构做标签；没有奖励，也没有试错。

数学和代码自带 **simulator**。

RL 的新战场：做题，对答案

$$\max_{\theta} E_{y \sim p_\theta}\left[r\left(y\right)\right] - \beta \mathrm{KL}\left(p_\theta \middle\| p_{\mathrm{ref}}\right)$$

奖励来自程序：答案匹配、单元测试、Lean 编译

Q：第四章讲什么？

受摄二体问题	0
天体运动中的共振现象	**1**
有序与混沌运动	0

采样 3 条 → 验证器判分 → 更新策略

“strawberry” 里有几个 r? s t r a w b e r r y GPT-4o 常答 2 → **o1 / R1 答 3** 先拆字母，再数

2024.09 o1 → 2025.01 R1 · DeepSeek moment

> 同一段文字，从“预测下一个词”变成“答对才给分”。

RL 的常见算法

PPO

策略梯度 + critic

RLHF 的经典选择

GRPO

组内相对优势

省掉 critic；数学/代码常用

RLOO

留一法基线

轻量，方差更小

DPO

直接学偏好

不走 RL 循环

采样 → 奖励 → 更新

PPO / GRPO / RLOO 是 RL；DPO 是偏好优化，常被归到 RLHF 谱系。

> 算法不同，循环相同：采样 → 打分 → 更新。

数学和代码的天然优势

	一般任务	数学 / 代码
奖励	人打分 / 奖励模型	**程序判定**
成本	贵、慢、有噪声	**几乎为零**
供给	有限	**可无限生成**
反馈	一个分数	**具体错误位置**

01 分解

函数、模块 → 子目标

02 验证

测试 → 自己检查

03 调试

报错 → 定位 → 修正

04 长程约束

类型、变量、控制流 → 不丢状态

> 反馈越明确，试错越便宜，RL 越有效。

从会说话到会干活

会说话 · 2023

ChatGPT

指令微调 + RLHF

prompt / system

会思考

CoT

写出中间步骤

context engineering · pruning

会干活

Agent

tool calling · loop

memory

> 每一步都在解决前一步留下的问题。

2023：从补全到对话

user 用一句话解释希尔区。

assistant 给定能量下，天体可能出现的空间区域。

指令微调

学会对话格式：该回答，而不是续写

RLHF

学会人觉得好的回答：有用、无害

但仍然不会

不会用工具，不会记事，不会验证

> 会说话，是训练出来的。

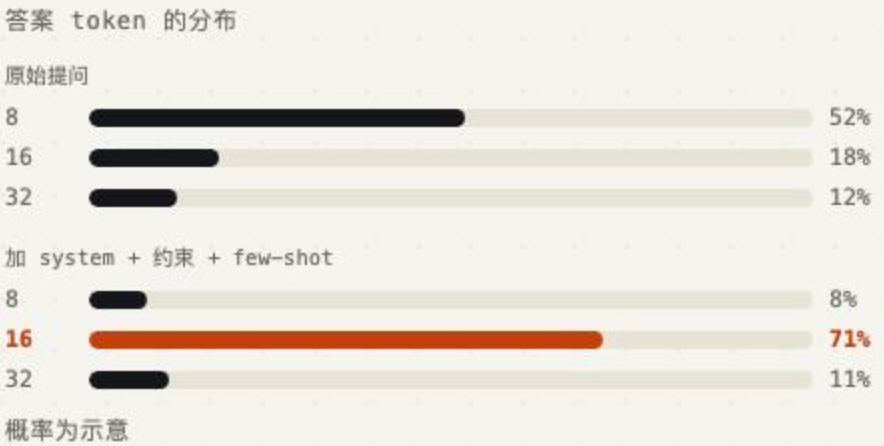
Prompt：用内容改变分布

SYSTEM · 角色
你是天体力学助教。

CONSTRAINTS · 约束
先写公式，再代入，最后给数值。

USER · 任务
周期 2 年，半长轴变为 4 倍，周期变成多少?

FEW-SHOT · 例子
a 变为 9 倍 → T 变为 27 倍
a 变为 16 倍 → T 变为 64 倍



$$p_\theta(y \mid x) \rightarrow p_\theta(y \mid x, c, e)$$

p(16 | x) = 0.18
p(16 | x, c, e) = 0.71

> 加内容，就是在改**条件概率**。

CoT：把思考写出来

范围：言语与逻辑推理；算术是另一件事。

问题

周期 2 年，半长轴变为 4 倍，周期变成多少?

直接答

8 年 × —— 一个没有计划的数字

COT · 计划

$T \propto a^{3/2}$；$T = 2 \times 4^{3/2}$

计划 ✓ · 算术下一步

工具 · 计算

```
run_python("2 * 4**1.5") → 16.0 ✓
```

COT 做什么

把文字题拆成计划：识别、回忆、设关系

剩下什么

算术：另一件适合工具的事

本质

显式步骤；精确数字是另一项能力

> CoT 负责写计划；程序负责算数。

Context window 的膨胀



贵

按 token 计费；长上下文预填更慢

慢

prefill 与注意力成本上升

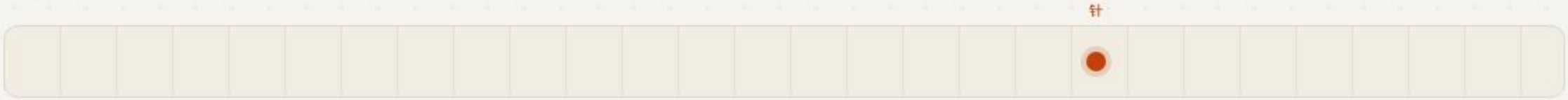
稀释

中间信息被忽略；信号被噪声淹没

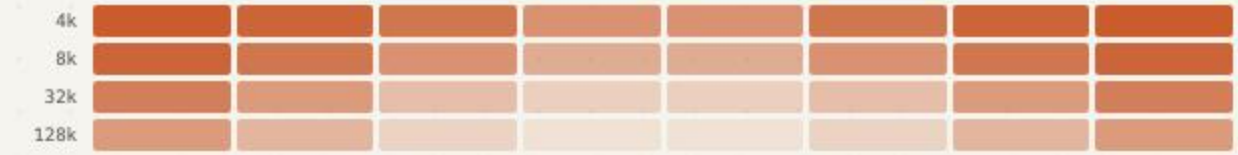
> 信息密度比窗口长度更重要。

Context 是有限的

大海捞针：在长文里藏一句话



检索准确率（示意）



横轴：针的位置 0 → 100%

怎么做

藏一句话，换长度、换位置，问它是什么

结果

越长越难；中间位置最差

含义

窗口大 ≠ 找得到

> 检索准确率才是关键。

needle in a haystack

Context engineering：把窗口当工作台

窗口里装什么

- system：规则
- 历史：前几轮
- 检索：相关文档
- 工具：返回结果
- 当前：这次的问题

太长

检索，只放相关的

太散

结构化，分块标注

中间丢失

关键信息放首尾

超窗

压缩、摘要、子 agent

> 上下文越相关越好。

CoT pruning：只写值得写的思考

一条长思维链

读题 → 设变量 → 展开 → ~~重复展开~~ → 分类讨论 → ~~无关分支~~ → 回到主线 → 代入 → 验算 → 答案

剪枝

去掉重复、无关的分支

蒸馏

用长 CoT 教短 CoT

摘要

长链压成短链

预算

限制思考 token，早停

> 思考也有预算。

Tool calling：算不准，就交给程序

```
工具：run_python(code: str) → str

call   run_python("print(2 * 4**1.5)")
obs    16.0

// T = 2 × 4**1.5 = 16
```

01 定义

工具签名：名字、参数、返回

02 调用

模型输出一段代码

03 执行

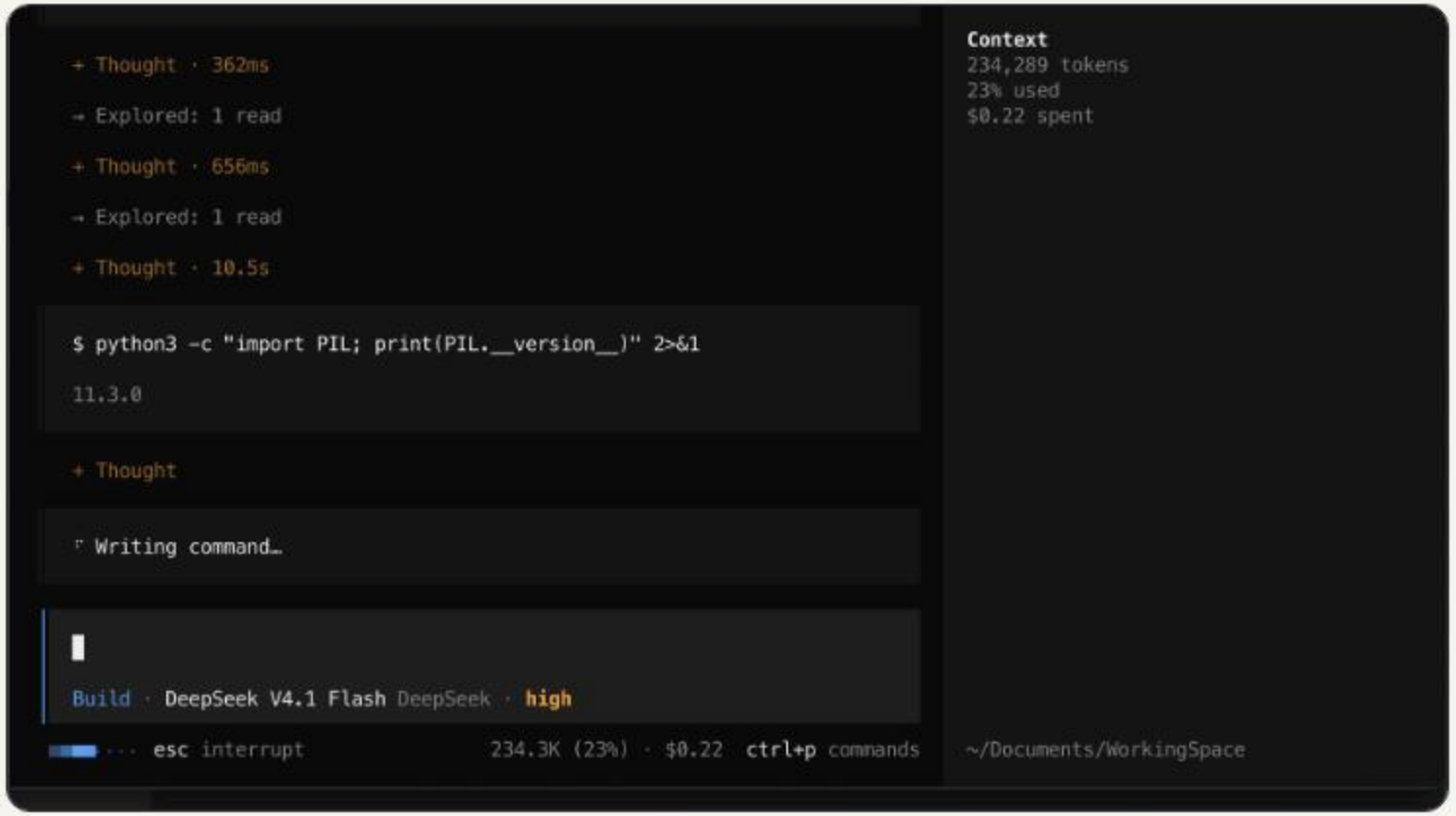
沙箱里真的跑

04 观察

结果回到上下文，继续

> 模型负责提议，程序负责算对。

Agent：模型被放进一个循环



01
循环
Thought → 工具调用 → 观察，重复

02
上下文
234K tokens · 23% used · $0.22

03
客户端
模型 + 工具 + 会话 + 账单

04
模型
DeepSeek V4.1 Flash · high

> 模型负责提议，客户端负责把提议变成**行动**。

Memory：信号与噪声

窗口里的内容

信号
任务 · 约束 · 决定 · 已验证事实

噪声
闲聊 · 重复 · 过期 · 原始日志

写入
只存决定与事实

检索
按当前任务取

压缩
摘要、合并

遗忘
过期即删

工作记忆
当前窗口；只放这次要用的

情节记忆
历史摘要；决定与未决问题

语义记忆
外部库 + 检索；按需取

程序记忆
system + tools；规则与能力

> 记忆的关键是取得准。

Harness 实例：Claude Code 代码审查

一个供学术同行参考的软件工程案例。

目标：**减少人工审查时间**，同时以回滚率与事故率把关。

01 · 提交之前

给出运行证据

- 执行测试、构建与界面检查
- 新上下文的 Agent 独立审查
- 加载项目约定与审查标准

02 · 分层检查

检查这次变更

- AI 审查与问题复核
- 安全审查与自动测试
- 按变更风险安排人工审批

03 · 发布与责任

控制失败影响

- 合并检查与分阶段发布
- 快速发现、快速回滚
- 每次变更由人承担责任

事故与回滚 → 经确认的评测案例 → 评估审查器改进 ↻

上下文资产：CLAUDE.md + REVIEW.md · 三轮后重新审视方案；后续审查聚焦新增变更。

把失败沉淀为**可复用的检查**，让审查经验持续**复利**。

来源：Osmani、Lunt、Wang · 2026-10-15 · How the Claude Code team does code review · 团队自述实践。

把模型放回系统里



内圈：一次调用 · 中圈：跨调用 · 外圈：工程边界

01	**LLM**	概率生成器：输入 token，输出下一个 token 的分布
02	**Prompt**	这次的条件：system / user / few-shot
03	**Context**	这次能看到的全部 token；窗口有限
04	**Memory**	窗口外的状态：写入 / 检索 / 压缩 / 遗忘
05	**Agent loop**	观察 → 行动 → 观察；状态放到外部
06	**Harness**	工具、沙箱、权限、验证、日志

> 模型是 CPU；prompt、context、memory、loop、harness 是运行时。

加速：从基本应用到金牌

2023 · 之前

基本应用

- 解释概念、查文献、写代码
- 做简单计算，经常算错
- 复杂证明仍要人
- 角色：助手

→ 约 19 个月

2025 · 之后

研究级

- IMO 2025 金牌级 35/42
- Lean 形式证明可验
- OpenAI 2026：372 题，平均 3 小时/题（报告）
- 角色：候选生成器

2024.07 **28/42** 银牌 → 2025.07 **35/42** 金牌级　12 个月，+7 分

> 这是一条阶梯：**三个台阶**。

from basic apps to research-level

时间线：三个台阶

2023–24

工具：搜索 + 验证

2023.12
FunSearch
LLM + evaluator + 进化搜索；cap set 新构造

2024.01
AlphaGeometry
神经符号；IMO 几何 25/30

2024.07
AlphaProof
Lean + RL；IMO 2024 银牌 28/42（+AG2）

2024–25

推理：RL + 测试时计算

2024.09
OpenAI o1
RL 学思考；test-time compute

2025.01
DeepSeek-R1
RLVR；推理行为涌现；开放权重

2025.05
AlphaEvolve
进化编码 agent；4×4 复矩阵乘法 48 次

2025–26

规模：从 benchmark 到开放问题

2025.07
IMO 2025
金牌级 35/42；自然语言、无工具

2025
形式化金牌（声称）
Aristotle 等；Lean 可验

2026.05–10
OpenAI
单位距离反例；NS / 黎曼相关（声称）；372 个问题（报告）

> 每一次跃迁，都是**生成器 + 验证器 + 搜索**。

他们怎么说的

> 公开材料覆盖了方法的一部分；完整的训练配方仍未公开。注意一种不对称：有些结果验证容易，证明很难。下面是论文、博客和赛后声明里的**公开表述**（节选 / 意译）。

时间	系统 / 工作	公开表述	谱法
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	搜索
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	合成数据
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	形式化
2024.09	**o1**	"spend more time thinking through problems"	推理
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	进化搜索
2025.07	**IMO 2025**	"natural language ... no tools"	端到端
2026.09–10	**OpenAI**	"NS solution; 372 problems; 722 papers"	开放问题

> 关键词在变，底层动作没变：**提议、验证、再搜索**。

共同的模式：生成 → 验证 → 搜索

生成

LLM

候选证明、程序、构造

→

验证

Lean / 测试 / 数值

给出对错、分数、反例

→

搜索 / 学习

best-of-N · 树 · 进化 · RL

把信号变成下一步策略

↺ 回到生成器：下一轮更强

没有验证器

只能靠人看；无法规模化

没有搜索

一次生成；好答案靠运气

两者都有

概率被筛成正确，再写回模型

> 模型提供概率，**验证器提供梯度**。

一些猜测：大结果可能是怎么来的

以下机制假设，来自公开论文、博客和 benchmark 的交叉印证，标注为**猜测**。

01 数据

自动形式化 + 合成：非形式题转 Lean；拒绝采样留下可验证的。

02 训练

RLVR + 长 CoT：答案可验证，奖励才能规模化。

03 推理

Test-time search：best-of-N、树搜索、进化。

04 蒸馏

把搜索写回模型：下一次不用搜那么久。

05 工程

Harness：并行、缓存、评测、回放。

06 赌注

验证：谁有更硬的 verifier，谁就有更快的飞轮。

OpenAI：372 题，平均 3 小时/题（报告）

3 小时/题，更像**并行采样 + 验证 + 搜索**。

还没解决的：从解题到做数学

已经发生

竞赛题

IMO 金牌级；自然语言、限时

形式证明

Lean 可验；不再依赖人眼

构造搜索

cap set、矩阵乘法、调度

开放问题

OpenAI 报告 372 个；筛选仍靠人

还没

选题

什么问题值得做

定义

新概念、新结构

长程

几个月的研究计划

品味

美、重要、可生长

> 解题可以搜索；**做数学需要方向**。

电机与流水线

Eric Schmidt 讲过一个故事；AI 正在经历同一件事。

电机刚被发明的时候，工厂主的做法很直觉：把中央蒸汽机拆掉，换一台电机，其他一切不变。效率提高了大约 30%。

过了几十年，有人意识到电机可以做得很小。每台机器可以有自己的电机。工厂可以围绕生产流程本身来设计；动力源变成每台机器自己的事。

流水线诞生了。效率提高了几十倍。

30%

单机模式 · Prompting Harder
换窗口即归零 · 无法积累

10×+

流水线 · Context Engineering
沉淀规则与模板 · 持续复利

30% 和几十倍的差距，是**可积累的工作方式**。

> 模型是租来的，上下文与系统才是自己的：别只在旧步骤里「问一下 AI」，把规则、工具与检验沉淀为长期资产。

收益在**布局与复利**。

算法工程师的困境

这个问题在一年前出现；数学正在经历同一件事。

“如果模型会写代码，我的工作是什么？”

之前

写算法

- 实现、调试、优化
- 对代码负责
- 价值 = 交付代码

→ 约一年

现在

指挥搜索

- 定义问题
- 设计 harness
- 验证、对系统负责

> 工作从「产出答案」转向**「指挥搜索」**。

从网页对话框到工作目录

严肃科研不能停留在薄外壳；文件即接口，规则可移植。

L1 · WRAPPER

网页对话

一问一答，手工复制

无文件访问权 · 无法沉淀

没有真实工具

模型口头描述，人类肉身执行

L2–3 · COPILOT

IDE 编辑器

单点补全与生成

上下文局限于当前打开文件

更快的打字机

未跳出旧循环，依然人工驱动

L4–5 · AGENT LOOP

工作目录 Agent

工具集成与子智能体

Python 辛积分 · 撮动展开 · 隔离审查

自主闭环与版本可控

假说 → 实验 → 守恒校验 → git 归档

- OpenCode
- Claude Code
- Codex CLI
- Cursor
- Markdown SOP 协议

> 文件即接口；在 **本地工作目录** 沉淀可移植的科研资产。

换模型只需切一行配置；你积累的动力学检验器与 Skill 资产永久有效。

两次尝试

同样的目标，不同的治理方式：从放任到做 Coach。

OPENCLAW · 早期尝试

误当计算器 · 完全放任

- 试图让 agent 端到端全包写完论文。
- 早期模型弱、幻觉多，且无任何外部检验回路。
- 人类彻底退场，期待自动化奇迹。

教训：脱离验证与引导，单机必定崩溃。

CODEX · GPT-5.3–5.5 · ISTART

做 Coach · 建立验证回路

- 博后工作延伸；没时间手写，由我定义清晰目标（WHAT）。
- 不做逐句修改的 Fixer，做指出破绽的 Coach。
- 纠正荒谬推导：$Ax = b,\ b \neq 0 \to x = 0$ ✕
- 拒稿 3 次后，用 GPT-6.1 Sol 扫出深层隐患并修正。

成效：验证不对称性——Eval 比生成更容易。

> 不做救火的 Fixer，做定标准与反例的 **Coach**。

低垂的果实与长期复利

生成论文只是起点；可复利的资产是评估器与科研品味。

01 · 人类作为 COACH

定目标与反例

明确定义 WHAT，放手让 AI 探索 HOW；给出逻辑反例，拒绝人工逐行代修。

02 · 系统化的 EVAL

动力学守恒检验

好模型仍会卡壳。沉淀能量守恒、辛对称性校验集；机器负责计算，残差负责把关。

03 · 可复利资产

品味与评估集

单篇论文是易贬值的果实；持久复利的是知识库、代码 SOP 与形式化评测器。

OpenAI 可以批量抽卡爆破开放问题，但以前的猜想是写给人的。当产出边际成本归零，唯有**评估系统与逆向品味**真正复利。

> 下一个核心问题：哪个问题**值得机器与人类一起做**?

年轻人的机会

天体力学正在变成数据领域；稀缺的是找到信号的能力。



数据

Gaia、卫星跟踪、巡天、N 体模拟。

数据比手多。



手艺

去噪、建模、验证；agent 跑流水线。

新的杠杆。



信号

共振、异常、新轨道。

问题还是老的。

数据比手多，工具是新的，问题是老的。**这就是年轻人的机会。**

> 瓶颈在**信号**。

从下一个 token 到下一个定理

token → 概率 → 候选 → 验证 → 定理

概率负责广度

验证负责正确

人负责方向

> 下一个定理，仍然从**下一个 token** 开始。